Let's trace a complete, step-by-step micro-example of **Multi-Head Attention with 2 Heads** in a 3D vector space ($d = 3$).

Here is the exact pipeline you described:

$$\text{Input } (X) \longrightarrow \begin{cases} \textbf{Head 1}(W_{Q1}, W_{K1}, W_{V1}) \\ \textbf{Head 2}(W_{Q2}, W_{K2}, W_{V2}) \end{cases} \longrightarrow \text{Concatenate} \longrightarrow W_O \longrightarrow W_2 \longrightarrow \text{Final 3D Vector}$$

Step 1: The Input Matrix (X)

Let's use our position-aware input matrix X for our two words ("river" and "bank"), combining word embeddings and positional encodings:

$$X = \begin{bmatrix} 1.64 & 0.64 & 0.20 \\ 1.01 & -0.02 & 0.50 \end{bmatrix}$$

Step 2: Defining Weights for Head 1 and Head 2

Instead of one set of Q, K, V weights, we now have **two independent sets** (one for each head). Let's define simple 3×3 weight matrices for them:

- **Head 1 (e.g., focused on local syntax/identity):**

$$W_{Q1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad W_{K1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad W_{V1} = \begin{bmatrix} 0.9 & 0.1 & 0.0 \\ 0.1 & 0.8 & 0.1 \\ 0.0 & 0.2 & 0.9 \end{bmatrix}$$

- **Head 2 (e.g., focused on semantic context/relationships):**

$$W_{Q2} = \begin{bmatrix} 0.8 & 0.2 & 0.0 \\ 0.0 & 1.0 & 0.0 \\ 0.1 & 0.0 & 0.9 \end{bmatrix}, \quad W_{K2} = \begin{bmatrix} 0.9 & 0.0 & 0.1 \\ 0.0 & 1.0 & 0.0 \\ 0.0 & 0.1 & 0.9 \end{bmatrix}, \quad W_{V2} = \begin{bmatrix} 0.5 & 0.5 & 0.0 \\ 0.2 & 0.6 & 0.2 \\ 0.0 & 0.1 & 0.9 \end{bmatrix}$$

Step 3: Running Each Head Independently

Each head independently computes its own Queries, Keys, Values, and Softmax Attention scores:

1. **Head 1 Output (H_1):**
    - Computes its own attention weights and blends its V_1 vectors.
    - Produces a contextualized output matrix for the words: H_1 (size 2×3).
    - *Result for "bank" from Head 1:* $[\mathbf{0.91}, \mathbf{0.19}, \mathbf{0.45}]$
2. **Head 2 Output (H_2):**
    - Uses its unique weights (W_{Q2}, W_{K2}, W_{V2}) to look at the sentence from a different angle.
    - Computes its own attention map and blends its V_2 vectors.
    - Produces its own output matrix: H_2 (size 2×3).
    - *Result for "bank" from Head 2:* $[\mathbf{0.42}, \mathbf{0.51}, \mathbf{0.38}]$ (focusing on a different blend of features).

Step 4: Concatenation and the Common Output Matrix (W_O)

Now we take the outputs of both heads and **concatenate** them side-by-side.

- Since Head 1 is 2×3 and Head 2 is 2×3, concatenating them creates a wide matrix of size 2×6:

$$\text{Concat}(H_1, H_2) = [\, H_1 \mid H_2 \,] = \begin{bmatrix} \cdots & \cdots & \cdots & \vdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \vdots & \cdots & \cdots & \cdots \end{bmatrix} \quad \text{(Size: } 2 \times 6\text{)}$$

To bring the dimension back down to our model's standard size (3), we multiply this concatenated matrix by the **Common Output Projection Matrix (W_O)**, which is a 6×3 matrix:

$$\text{Attention Output} = \text{Concat}(H_1, H_2) \times W_O \quad \text{(Size: } 2 \times 3\text{)}$$

- W_O learns to mix the insights of Head 1 and Head 2 together into a single, unified 3D representation.

Step 5: Final Projection (W_2) to the Target 3D Vector Space

After passing through Feed-Forward Networks (FNN) inside the Transformer block, the data reaches the final projection layer W_2:

- W_2 takes the internal features and maps them into your final target space (such as your 3D Multiple Negatives Ranking vector space).
- *Final Result:* We get a clean, fully contextualized 3D vector for "bank" that successfully incorporates positional order, multi-head grammatical/semantic blending, and global projection!